

dashAI — Software Architecture Document (SAD)

Arsitektur perangkat lunak menurut ISO/IEC/IEEE 42010 dengan model C4

dashAI

2026-06-21

Daftar Isi

1	Pendahuluan	2
1.1	Tujuan dokumen	2
1.2	Ruang lingkup	2
1.3	Status demonstrasi & kejujuran rekayasa	3
1.4	Definisi & singkatan	4
2	Tujuan & batasan arsitektur	4
2.1	Sasaran arsitektur (<i>architectural drivers</i>)	4
2.2	Batasan (<i>constraints</i>)	6
2.3	Asumsi	6
3	Stakeholder & concern	7
4	C4 Level 1 — System Context	8
4.1	Aktor & sistem eksternal	9
4.2	Interaksi kunci	9
5	C4 Level 2 — Containers	9
5.1	Container klien (Browser PWA)	10
5.2	Container server (Next.js Functions @ Vercel)	11
5.3	Aliran antar-container	11
6	C4 Level 3 — Components (browser app + seal flow)	11
6.1	Komponen CV & Violation (klien)	13
6.2	Komponen seal/verify (server + klien)	14
6.3	Knowledge base hukum	14
7	Runtime view — siklus hidup bukti	14
7.1	Pipeline CV per-frame	15
7.1.1	Aturan <i>live</i> yang aktif	16
7.2	Alur penyegelan & verifikasi	17
7.3	Mesin keadaan pelanggaran	19
7.4	Pemicu otomatis kecelakaan	21
8	Data view	21
8.1	Model domain & batas kriptografis	21
8.1.1	EvidencePayload (yang ditandatangani)	23
8.1.2	SealedEvidence (envelope tertandatangani)	23

8.2	Penyimpanan & retensi	24
8.3	Aliran data & kanonikalisasi	24
9	Deployment view (Vercel)	25
9.1	Topologi	25
9.2	Konfigurasi & rahasia	26
9.3	Karakteristik <i>deployment</i>	26
10	Cross-cutting concerns	27
10.1	Keamanan	27
10.2	Privasi	29
10.3	Performa	30
10.4	Observability	30
11	Technology decisions & rationale	31
12	Quality scenarios	33
12.1	QS-1 Integritas (Security)	33
12.2	QS-2 Kemandirian verifikasi (Security/Trust)	33
12.3	QS-3 Privasi koordinat (Privacy)	33
12.4	QS-4 Ketahanan jaringan (Availability)	33
12.5	QS-5 Robustness model (Reliability)	33
12.6	QS-6 Akurasi yang jujur (Correctness)	33
12.7	QS-7 Provenans hukum (Maintainability/Trust)	33
12.8	QS-8 Anti-pemalsuan laporan (Security)	34
12.9	QS-9 Minimnya jejak waktu palsu (Integrity)	34
12.10	QS-10 Pluggability (Modifiability)	34
13	Lampiran — Ringkasan keterbatasan demo	34

1 Pendahuluan

1.1 Tujuan dokumen

Dokumen ini adalah *Software Architecture Document* (SAD) untuk **dashAI**, sebuah *AI dashcam* berbasis web yang mendeteksi pelanggaran lalu lintas secara real-time di dalam peramban, mengaitkan setiap pelanggaran ke pasal **UU No. 22 Tahun 2009 tentang Lalu Lintas dan Angkutan Jalan (LLAJ)**, lalu menyegel bukti secara kriptografis dengan tanda tangan **Ed25519** sehingga bersifat *tamper-evident* (terbukti-bila-diubah, **bukan** *tamper-proof*).

SAD ini disusun mengikuti kerangka **ISO/IEC/IEEE 42010:2011** (deskripsi arsitektur: *stakeholder*, *concern*, *viewpoint*, dan *view*) dan menyajikan arsitektur statis dengan model **C4** (Context, Container, Component) ditambah *runtime view*, *data view*, dan *deployment view*. Setiap pernyataan dalam dokumen ini dilandaskan pada kode sumber nyata pada repositori **Finerium/dashAI** (Apache-2.0).

1.2 Ruang lingkup

Cakupan dokumen ini adalah seluruh sistem perangkat lunak dashAI sebagaimana berjalan saat ini: aplikasi **Next.js 16 (App Router) + React 19 + TypeScript (strict) + Tailwind CSS v4** yang di-*deploy* di **Vercel** pada <https://dashai-mu.vercel.app>. Ini mencakup *pipeline computer-vision*

di sisi klien, mesin aturan pelanggaran, *knowledge base* hukum terverifikasi, lapisan penyegelan kriptografis di server, serta jalur pembuatan laporan PDF dan verifikasi mandiri.

Di luar cakupan: perangkat keras dashcam khusus (rencana pasca-investor), integrasi resmi ETLE/kanal kepolisian, dan *backend* model deteksi khusus (YOLOv8/ONNX, model helm/pelat/kabin) — semuanya tercatat sebagai bagian *roadmap*, bukan arsitektur saat ini.

1.3 Status demonstrasi & kejujuran rekayasa

dashAI saat ini berstatus **demonstrasi teknologi melalui kamera ponsel**, bukan instrumen penegakan hukum resmi. Dua keterbatasan ditegaskan secara eksplisit dalam arsitektur agar tidak menyesatkan:

1. **Tamper-evident, bukan tamper-proof.** Tanda tangan membuktikan bahwa isi laporan tidak berubah satu *byte* pun sejak disegel server pada waktu tertera; ia **tidak** membuktikan bahwa kamera benar-benar menyaksikan peristiwa di dunia nyata (frame berasal dari klien yang dapat diarahkan ke layar). Hal ini ditulis langsung di komentar `lib/crypto/sign.ts`.
2. **Estimasi kecepatan kendaraan lain bersifat perkiraan.** Kecepatan diri sendiri (*self*) akurat karena berasal dari GPS dibandingkan maxspeed OSM; kecepatan kendaraan lain (*other*) adalah estimasi dari perubahan skala *bounding-box* dan *optical flow* dengan rentang galat.

1.4 Definisi & singkatan

Istilah	Penjelasan
LLAJ	Lalu Lintas dan Angkutan Jalan (UU No. 22 Tahun 2009)
PDP	Pelindungan Data Pribadi (UU No. 27 Tahun 2022)
C4	Model diagram arsitektur: Context, Container, Component, Code
CV	<i>Computer Vision</i>
IoU	<i>Intersection over Union</i> (metrik tumpang-tindih kotak deteksi)
coco-ssd	Model deteksi objek TensorFlow.js (varian <code>lite_mobilenet_v2</code>)
APILL	Alat Pemberi Isyarat Lalu Lintas (mis. lampu merah)
ETLE	<i>Electronic Traffic Law Enforcement</i>
Subject	Subjek pelanggaran relatif pemilik dashAI: <code>self</code> (diri sendiri) atau <code>other</code> (orang lain)
Seal	Proses menyegel payload bukti: kanonikalisasi $\rightarrow$ hash SHA-256 $\rightarrow$ tanda tangan Ed25519
Payload	EvidencePayload — <i>exact</i> JSON yang ditandatangani server; batas kriptografis sistem

2 Tujuan & batasan arsitektur

2.1 Sasaran arsitektur (*architectural drivers*)

Arsitektur dashAI dibentuk oleh sejumlah pendorong utama yang langsung tercermin dalam kode:

1. **Privacy-by-design / local-first.** Bukti disimpan lebih dulu di perangkat pemilik (IndexedDB, lihat `lib/evidence/store.ts`). Server hanya melihat payload ketika pengguna **secara eksplisit** menyegel/melaporkan sebuah event. Deteksi wajah memakai MediaPipe BlazeFace untuk **melokalkan-dan-memburamkan**, bukan mengenali identitas (`lib/cv/face.ts`).
2. **Inferensi di tepi (*edge inference*).** Seluruh CV berjalan di peramban (TensorFlow.js + WebGL, MediaPipe WASM/GPU, Tesseract.js). Tidak ada *video upload*; tidak ada *backend* GPU.

3. **Integritas bukti terverifikasi-independen.** Penandatanganan Ed25519 di server dengan **satu** rahasia (DASHAI_SIGNING_KEY); kunci publik diterbitkan via /api/public-key sehingga verifikasi tidak perlu memercayai UI dashAI.
4. **Provenans hukum, bukan halusinasi.** Sitasi UU berasal dari *knowledge base* yang digenerate oleh *workflow* riset multi-agent dengan **verifikasi adversarial 3-voter** (lib/legal/citations.ts, artefak docs/research/phase0-research.json).
5. **Degradasi anggun (*graceful degradation*).** Model sekunder (face, pose) dibungkus try/catch agar kegagalan *load*/inferensi tidak menghentikan *live view* (lib/cv/pipeline.ts). Pencarian konteks jalan ke Overpass mengembalikan null pada kegagalan jaringan apa pun (lib/geo/osm.ts).
6. **Tanpa biaya operasional pihak ketiga.** Tidak ada *API key* berbayar: model dimuat dari CDN publik (TF Hub, jsDelivr, GCS); konteks jalan dari OpenStreetMap Overpass (tanpa kunci).
7. **Pluggability.** Mesin pelanggaran adalah *rule set* berkunci ViolationKey; menambah pelanggaran = tambah entri katalog + sitasi + aturan detektor (lib/violations/engine.ts, lib/legal/catalog.ts). Detektor objek di balik antarmuka ObjectDetector agar coco-ssd dapat ditukar ke YOLOv8/ONNX tanpa menyentuh *pipeline*.

2.2 Batasan (*constraints*)

#	Batasan	Sumber / implikasi
C-1	<i>Deploy</i> di Vercel; <i>serverless functions</i> memakai <code>runtime = "nodejs"</code>	<code>app/api/*/route.ts</code> (<code>export const runtime = "nodejs"</code>) — Ed25519 & PDF butuh kapabilitas Node
C-2	Hanya satu rahasia: <code>DASHAI_SIGNING_KEY</code> (seed Ed25519 32-byte hex)	<code>lib/crypto/keys.ts</code> ; tanpa env, sistem memakai kunci DEV bertanda <code>dev-...</code>
C-3	Model & weights tidak di- <i>bundle</i> ; dimuat saat <i>runtime</i> dari CDN	<code>lib/cv/detector.ts</code> , <code>face.ts</code> , <code>pose.ts</code> — memungkinkan ekspor statis aset
C-4	<code>@react-pdf/renderer</code> harus eksternal terhadap <i>bundler</i> server	<code>next.config.ts</code> → <code>serverExternalPackages:</code> <code>["@react-pdf/renderer"]</code>
C-5	Privasi koordinat: lat/lng dibulatkan ke 4 desimal (~11 m) sebelum dikirim ke Overpass	<code>lib/geo/osm.ts</code> (<code>toFixed(4)</code>)
C-6	Deteksi <i>live</i> terbatas pada pelanggaran dengan sinyal CV/GPS yang sah	<code>lib/violations/engine.ts</code> — <code>helm/pelat/lampu/kabin</code> dikatalogkan, belum <i>live</i>
C-7	TypeScript <i>strict</i> ; tipe domain berpusat pada <code>EvidencePayload</code> sebagai batas krypto	<code>lib/evidence/types.ts</code>

2.3 Asumsi

- Peramban target mendukung `getUserMedia`, `Geolocation`, `DeviceMotion`, `WebGL/WASM`, dan `IndexedDB` (peramban ponsel modern).
- Pengguna memberikan izin kamera, lokasi, dan (pada iOS 13+) gerak perangkat secara eksplisit (`lib/sensors/motion.ts` → `requestMotionPermission`).
- Layanan publik (Overpass, CDN model) tersedia; bila tidak, sistem terdegradasi, tidak gagal total.

3 Stakeholder & concern

Mengikuti ISO/IEC/IEEE 42010, arsitektur dideskripsikan melalui *stakeholder* dan *concern* yang ditangani oleh *view* terkait.

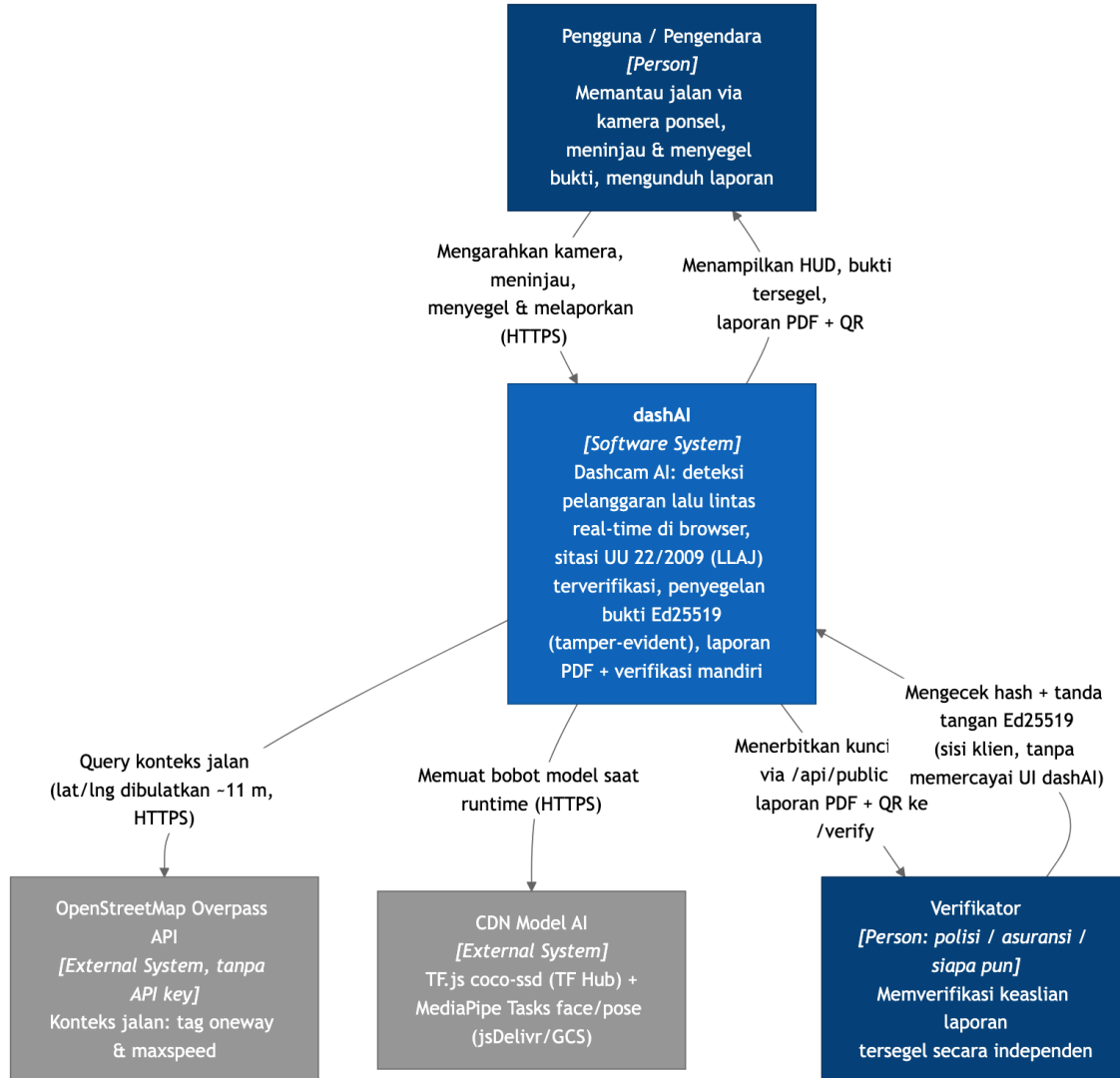
Stakeholder	Peran	Concern utama	View yang menjawab
Pengguna / Pengendara	Mengarahkan kamera, meninjau, menyegel, mengunduh laporan	Privasi, akurasi deteksi, perlindungan diri dari fitnah, kemudahan	Context, Container, Runtime, Cross-cutting (privasi)
Verifikator (polisi / asuransi / siapa pun)	Memverifikasi keaslian laporan secara independen	Integritas bukti, verifikasi tanpa UI, mempercayai kemandirian	Runtime (lifecycle), Component (seal/verify), Cross-cutting (security)
Pengembang / Maintainer	Mengembangkan & memelihara sistem	Modularitas, <i>pluggability</i> , kejelasan batas, <i>strict types</i>	Container, Component, Technology decisions
Operator / DevOps	Men- <i>deploy</i> & mengelola rahasia	Manajemen kunci, <i>runtime</i> , ketiadaan <i>state</i> server	Deployment, Cross-cutting (security/observability)
Penasihat hukum / Ahli LLAJ	Menjamin sitasi pasal benar	Provenans sitasi, konsistensi pasal, verifikasi 3-voter	Data view, Technology decisions
Otoritas perlindungan data	Memastikan kepatuhan PDP	Minimalisasi data, <i>local-first</i> , deteksi vs pengenalan	Cross-cutting (privasi)
Investor / Pemangku bisnis	Menilai kelayakan & arah produk	Keterbatasan demo yang jujur, <i>roadmap</i> ke produksi	Pendahuluan, Quality scenarios

Concern lintas-stakeholder yang menjadi tulang punggung arsitektur:

- **Integritas vs. attestation.** Sistem membuktikan *non-tampering* isi, bukan *witnessing* realitas. Pemisahan ini dijaga tegas di tipe EvidencePayload dan komentar `sign.ts/verify.ts`.
- **Minimalisasi data.** Server tidak menyimpan *state*; verifikasi *self-contained* (QR + URL membawa seluruh *envelope*, lihat `app/api/report/route.ts`).
- **Kemandirian verifikasi.** `verifySealed` berjalan identik di klien dan server (`lib/crypto/verify.ts`), sehingga kepercayaan tidak bergantung pada `route/api/verify`.

4 C4 Level 1 – System Context

Diagram konteks menempatkan dashAI di antara dua aktor manusia (Pengguna dan Verifikator) dan dua sistem eksternal tanpa *API key* (OpenStreetMap Overpass dan CDN model AI).



Gambar 1: C4 Level 1 – System Context

4.1 Aktor & sistem eksternal

Elemen	Tipe	Peran
Pengguna / Pengendara	Person	Memantau jalan via kamera ponsel; meninjau, menyegel, dan mengunduh laporan
Verifikator	Person (polisi / asuransi / siapa pun)	Memverifikasi keaslian laporan tersegel secara independen
dashAI	Software System	Deteksi pelanggaran real-time di peramban, sitasi UU 22/2009 terverifikasi, penyesel Ed25519, laporan PDF + verifikasi mandiri
OpenStreetMap API	External System (tanpa kunci)	Konteks jalan: tag oneway & maxspeed
CDN Model AI	External System	TF.js coco-ssd (TF Hub) + MediaPipe Tasks face/pose (jsDelivr/GCS)

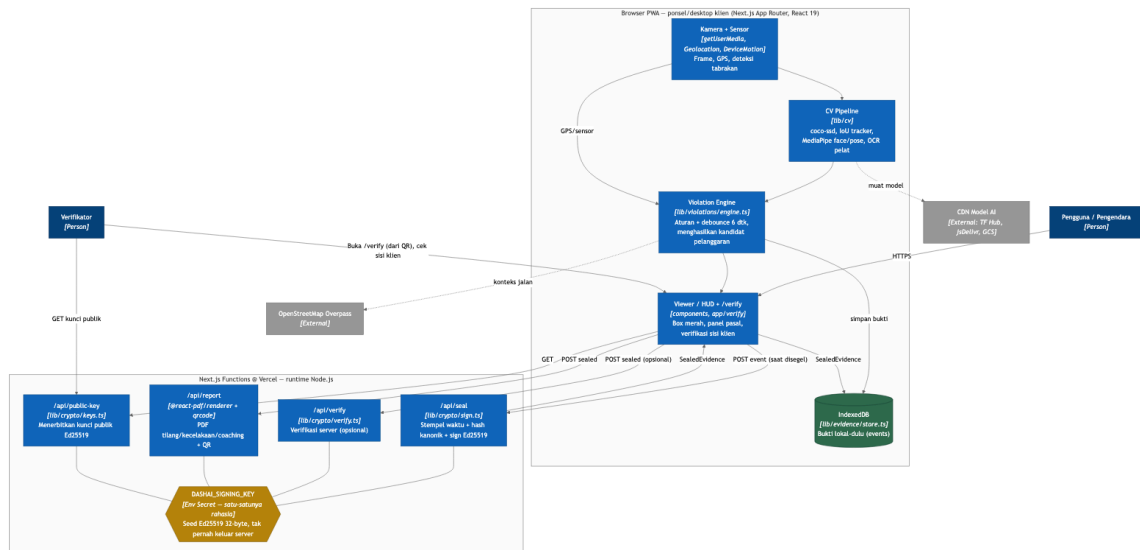
4.2 Interaksi kunci

- Pengguna **mengarahkan kamera, meninjau, menyegel & melaporkan** lewat HTTPS; dashAI **menampilkan HUD, bukti tersegel, dan laporan PDF + QR**.
- dashAI **mengkueri konteks jalan** ke Overpass dengan koordinat **dibulatkan ~11 m** (privasi), dan **memuat bobot model saat runtime** dari CDN.
- dashAI **menerbitkan kunci publik** via /api/public-key serta laporan PDF + QR ke /verify; Verifikator **mengecek hash + tanda tangan Ed25519 di sisi klien** tanpa memercayai UI dashAI.

Catatan arsitektural: tidak ada basis data eksternal, tidak ada penyimpanan video di server, dan tidak ada layanan AI pihak ketiga berbayar — sebuah konsekuensi langsung dari prinsip *edge inference* dan *local-first*.

5 C4 Level 2 — Containers

Pada level container, dashAI terbelah menjadi **Browser PWA** (klien) dan **Next.js Functions @ Vercel** (server *runtime* Node.js), ditambah dua sistem eksternal.



Gambar 2: C4 Level 2 – Containers

5.1 Container klien (Browser PWA)

Container	Sumber kode	Tanggung jawab
Kamera + Sensor	getUserMedia, lib/geo/location.ts, lib/sensors/motion.ts	Frame video, GPS (lat/lng, speedMps, headingDeg), deteksi tabrakan/rem keras
CV Pipeline	lib/cv/	coco-ssd, IoU tracker, MediaPipe face/pose, OCR pelat (Tesseract.js)
Violation Engine	lib/violations/engine.ts	Aturan + <i>debounce</i> 6 detik → menghasilkan ViolationCandidate
Viewer / HUD + /verify	components/, app/verify, app/page.tsx	Box merah, panel pasal, verifikasi sisi klien
IndexedDB	lib/evidence/store.ts (idb)	Bukti <i>local-first</i> (object store events, index by-time)

Status sesi *live* dikelola dengan **zustand** (lib/state/session.ts): status pipeline, FPS, jumlah deteksi, geo/road/ego-speed, dan daftar event.

5.2 Container server (Next.js Functions @ Vercel)

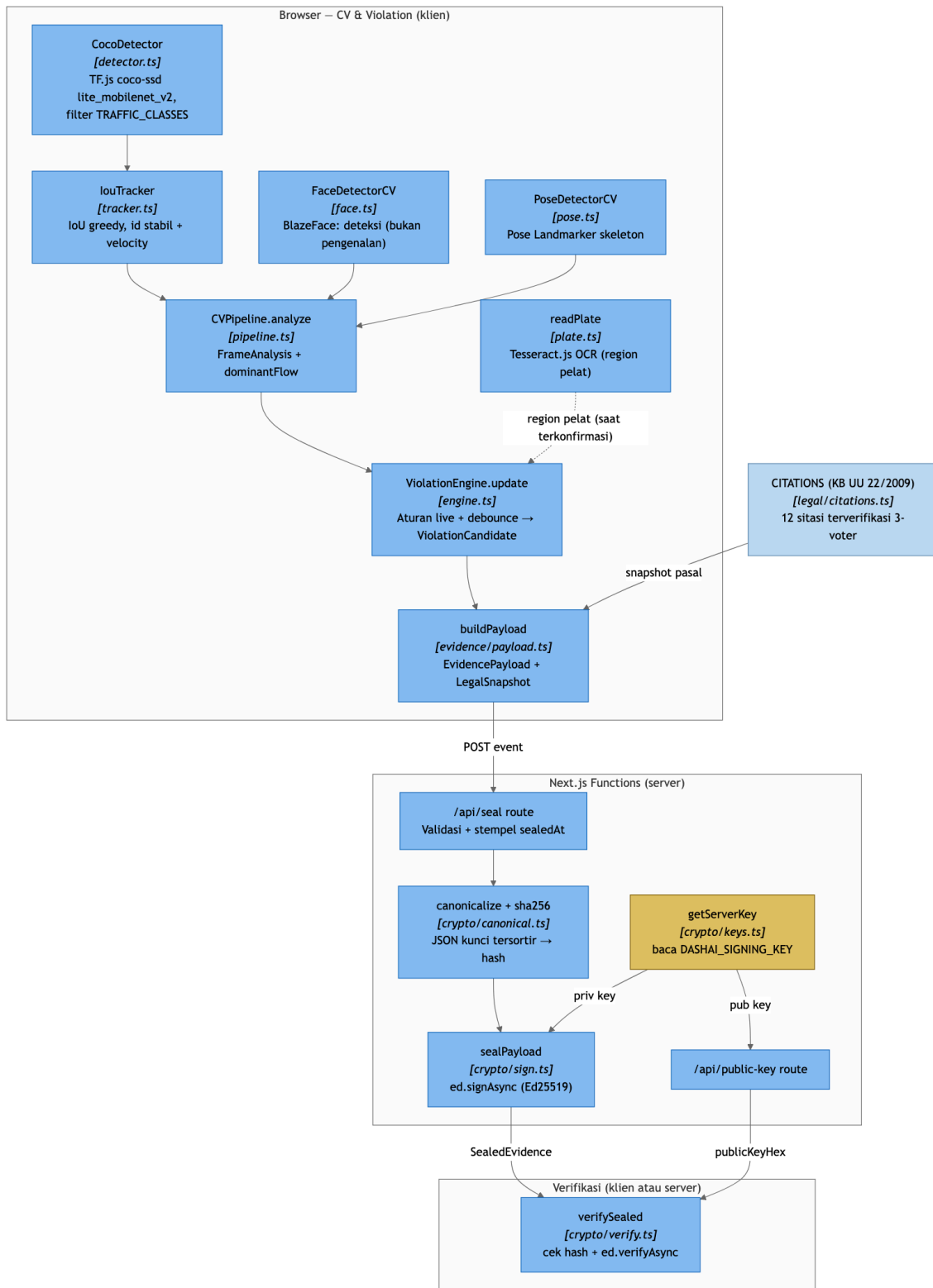
Endpoint	Sumber	Tanggung jawab	Runtime
POST /api/seal	lib/ crypto/sign.ts, lib/evidence/ payload.ts	Validasi event → stempel sealedAt server → bangun payload + snapshot pasal → SHA-256 kanonik → tanda tangan Ed25519	nodejs
POST /api/verify	lib/crypto/ verify.ts	Verifikasi server (opsional; kepercayaan tak bergantung padanya)	nodejs
POST /api/report	@react-pdf/ renderer + qrcode	Verifikasi ulang → render PDF (tilang/kecelakaan/ coaching) + QR ke / verify	nodejs
GET /api/public-key	lib/crypto/keys.ts	Menerbitkan kunci publik Ed25519 (Cache-Control: public, max- age=3600)	nodejs
DASHAI_SIGNING_KEY	Env Secret	Seed Ed25519 32- byte; satu-satunya rahasia , tak pernah keluar server	—

5.3 Aliran antar-container

Frame mengalir Kamera → CV Pipeline → Violation Engine → Viewer; GPS/sensor masuk langsung ke *engine*; bukti disimpan ke IndexedDB. Saat pengguna menyegel, Viewer **POST event** ke /api/seal dan menerima SealedEvidence, lalu menyimpannya kembali ke IndexedDB serta meneruskannya (opsional) ke /api/report dan /api/verify. Verifikator melakukan GET kunci publik dan membuka /verify (dari QR) untuk pemeriksaan sisi klien. Keempat *route* server hanya terhubung ke satu rahasia env.

6 C4 Level 3 — Components (browser app + seal flow)

Diagram komponen memerinci dua subsistem yang menjadi inti dashAI: **CV & Violation** (klien) dan **alur seal/verify** (server + klien), dengan *knowledge base* hukum sebagai sumber data.



Gambar 3: C4 Level 3 – Components

6.1 Komponen CV & Violation (klien)

Komponen	File	Detail teknis
CocoDetector	lib/cv/detector.ts	TF.js coco-ssd lite_mobilenet_v2; deteksi top-20, skor $\geq$ 0.45; filter TRAFFIC_CLASSES; bbox dinormalkan 0..1
IouTracker	lib/cv/tracker.ts	IoU greedy (ambang 0.3), maxMissed = 8; id stabil + velocity (vx,vy) + age
FaceDetectorCV	lib/cv/face.ts	MediaPipe BlazeFace short- range, minDetectionConfidence 0.5; deteksi , bukan pengenalan
PoseDetectorCV	lib/cv/pose.ts	MediaPipe Pose Landmarker lite, numPoses 4; skeleton (garis manusia)
readPlate	lib/cv/plate.ts	Tesseract.js OCR pada region pelat; regex pelat Indonesia; hanya saat terkonfirmasi
CVPipeline.analyze	lib/cv/pipeline.ts	Orkestrasi per-frame → FrameAnalysis + dominantFlow; model sekunder <i>graceful</i>
ViolationEngine.update	lib/violations/engine.ts	Aturan <i>live</i> + <i>debounce</i> per (violation, track) → ViolationCandidate
buildPayload	lib/evidence/payload.ts	Merangkai EvidencePayload + LegalSnapshot dari KB sitasi

6.2 Komponen seal/verify (server + klien)

Komponen	File	Detail
/api/seal route	app/api/seal/route.ts	Validasi kelengkapan event, <i>clamp</i> confidence ke 0..1, stempel sealedAt = Date.now()
canonicalize + sha256	lib/crypto/canonical.ts	JSON kunci tersortir rekursif (drop undefined) → SHA-256 hex
sealPayload	lib/crypto/sign.ts	ed.signAsync (Ed25519) atas byte kanonik; import "server-only"
getServerKey	lib/crypto/keys.ts	Baca DASHAI_SIGNING_KEY (regex <code>^[0-9a-fA-F]{64}\$</code>); fallback DEV; <i>cache</i>
/api/public-key route	app/api/public-key/route.ts	Terbitkan publicKeyHex, keyId, algorithm, isDev
verifySealed	lib/crypto/verify.ts	Cek (1) hash kanonik cocok, (2) ed.verifyAsync; identik di klien & server

6.3 Knowledge base hukum

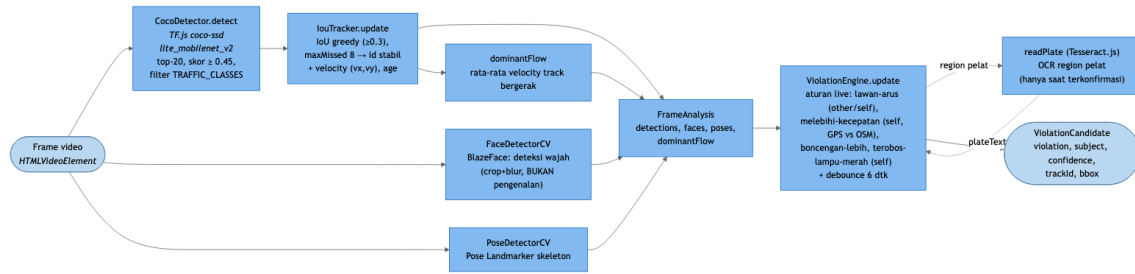
CITATIONS (lib/legal/citations.ts) adalah *record* berkunci ViolationKey berisi **12 sitasi terverifikasi 3-voter** untuk UU 22/2009. Saat buildPayload dijalankan, sitasi yang relevan di-*snapshot* menjadi LegalSnapshot (uu, pasal, ayat, sanksi) dan ikut ditandatangani — sehingga laporan mencantumkan pasal persis pada saat penyegelan.

Aliran: CocoDetector → IouTracker → CVPipeline; FaceDetectorCV & PoseDetectorCV → CVPipeline; CVPipeline → ViolationEngine; readPlate memasok plateText saat terkonfirmasi; ViolationEngine → buildPayload; CITATIONS → buildPayload; buildPayload → POST → /api/seal → canonicalize → sealPayload; getServerKey memasok *private key* ke sealPayload dan *public key* ke /api/public-key; sealPayload → SealedEvidence → verifySealed ← publicKeyHex.

7 Runtime view — siklus hidup bukti

Bagian ini menyajikan *runtime view* untuk skenario inti: dari frame kamera hingga bukti tersegel yang dapat diverifikasi siapa pun.

7.1 Pipeline CV per-frame



Gambar 4: Pipeline CV (frame → candidate)

Tiap frame `HTMLVideoElement` diproses sebagai berikut (`CVPipeline.analyze`):

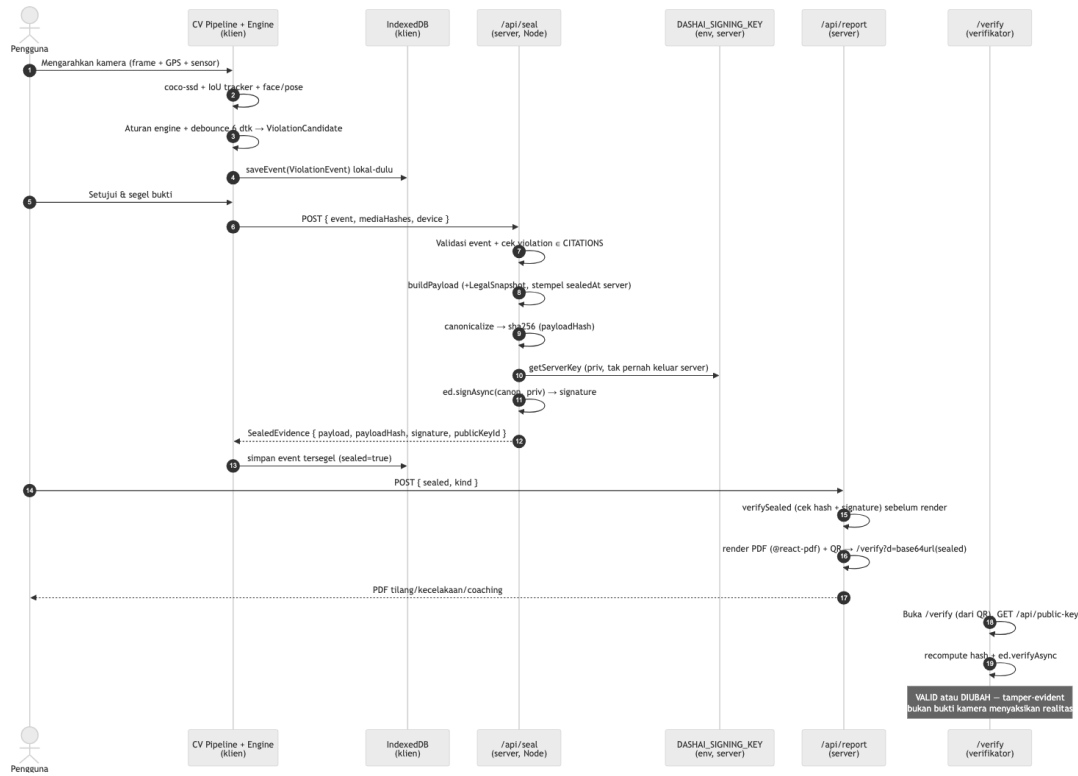
1. `CocoDetector.detect` → deteksi mentah (top-20, skor ≥ 0.45 , hanya `TRAFFIC_CLASSES`).
2. `IouTracker.update` → asosiasi IoU greedy (≥ 0.3), id stabil, hitung velocity (v_x, v_y) dan age; *age out* track yang hilang setelah `maxMissed` = 8.
3. `FaceDetectorCV` & `PoseDetectorCV` (opsional, *graceful*) → kotak wajah (untuk blur) dan skeleton.
4. `dominantFlow` → rata-rata velocity track yang bergerak (ambang > 0.01), menjadi acuan arah arus dominan.
5. Hasil dirangkum menjadi `FrameAnalysis { detections, faces, poses, dominantFlow }`.
6. `ViolationEngine.update` menerapkan aturan *live* + *debounce* 6 detik; saat sebuah pelanggaran terkonfirmasi, `readPlate` (Tesseract.js) dipanggil **hanya** pada region pelat untuk menghasilkan `plateText`. Output: `ViolationCandidate { violation, subject, confidence, trackId, bbox }`.

7.1.1 Aturan *live* yang aktif

Aturan	Subject	Sinyal & ambang (dari engine.ts)
lawan-arus (other)	other	dominantFlow valid, ≥ 3 kendaraan bergerak; <i>cosine similarity</i> track vs flow < -0.6 ; age ≥ 4
lawan-arus (self)	self	OSM oneway + bearingDeg; $ \text{angleDelta}(\text{heading}, \text{bearing}) > 120^\circ$; ego-speed ≥ 8 km/jam
melebihi-kecepatan (self)	self	GPS egoSpeedKmh $> \text{maxspeedKmh} + 5$ (akurat); confidence 0.95
boncengan-lebih	other	≥ 3 orang dengan pusat bbox di dalam bbox satu motorcycle (age ≥ 4)
terobos-lampu-merah (self)	self	trafficLightState === "red" dan ego-speed ≥ 10 km/jam

Pelanggaran lain (helm, pelat, lampu, sabuk, ponsel kabin) **dikatalogkan** di lib/legal/catalog.ts dengan *detection tier* (core/secondary/cabin) dan ditampilkan melalui *dataset* demo; deteksi penuh menuntut model khusus / kamera kabin (lihat *roadmap*).

7.2 Alur penyegelan & verifikasi



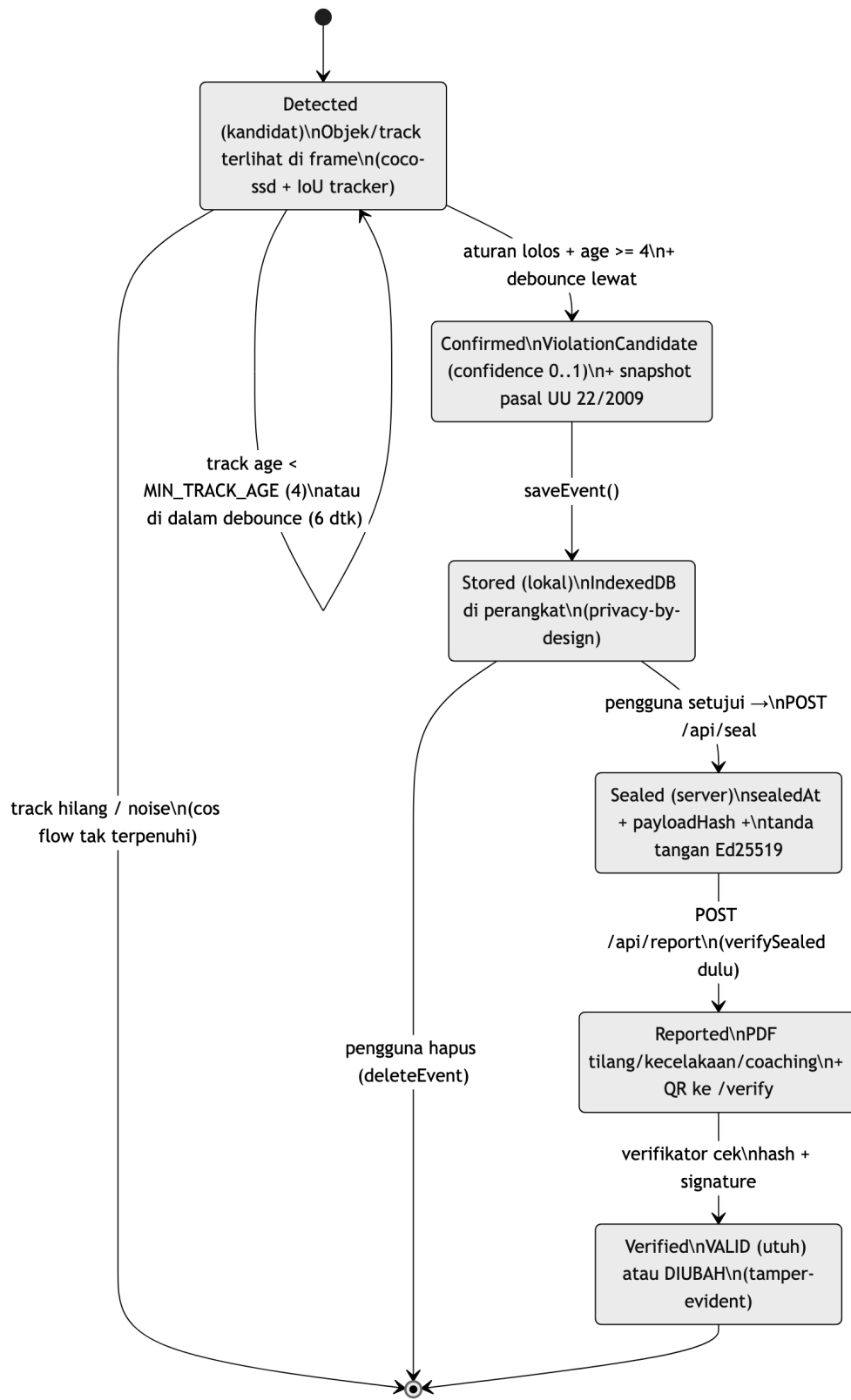
Gambar 5: Urutan penyegelan bukti

Skenario “menyegel bukti” (lihat juga `app/api/seal/route.ts`, `lib/crypto/*`):

1. **Kamera (klien)** memasok frame + GPS + sensor ke **Engine (klien)**.
2. **Engine** menerapkan deteksi + aturan; pelanggaran terkonfirmasi → membangun event.
3. **Klien** → **POST /api/seal** membawa event (id, jenis, plat, kecepatan, lokasi) plus mediaHashes dan device (opsional).
4. **Server** memvalidasi kelengkapan (id, violation, subject, capturedAt), menolak violation di luar CITATIONS, meng-*clamp* confidence ke 0.1, lalu **men-stempel sealedAt = waktu server** (otoritatif, tak pernah dipercaya dari klien).
5. **buildPayload** merangkai EvidencePayload (schema: "dashai.evidence.v1") + LegalSnapshot dari KB.
6. **sealPayload**: canonicalize (JSON kunci tersortir) → sha256Hex0fString → **ed.signAsync** dengan *private key* yang **tak pernah keluar server**.
7. **Server** → **klien**: SealedEvidence { payload, algorithm: "Ed25519", publicKeyId, payloadHash, signature, sealedAt }.
8. Klien menyimpan ke IndexedDB dan (opsional) memanggil **/api/report** → PDF. **/api/report memverifikasi ulang envelope**; jika tidak valid, mengembalikan 422 dan **tidak** membuat laporan. QR + URL menyandikan seluruh *envelope* (base64url) sehingga verifikasi *self-contained*.

9. **Verifikator** mengambil kunci publik (GET /api/public-key), lalu `verifySealed` memeriksa (a) hash kanonik cocok dan (b) tanda tangan Ed25519 valid → hasil **VALID** atau **DIUBAH** (*tamper-evident*).

7.3 Mesin keadaan pelanggaran



Gambar 6: Mesin keadaan pelanggaran

Daur hidup sebuah event: **terdeteksi** → **terkonfirmasi (lolos *debounce* & *age*)** → **tersimpan lokal** → **tersegel (*sealed* = *true*, *seal* terisi)** → **dilaporkan (PDF) / diverifikasi**. *Debounce* per (violation, trackId|subject) selama 6 detik mencegah satu pelanggaran berulang dilaporkan tiap frame.

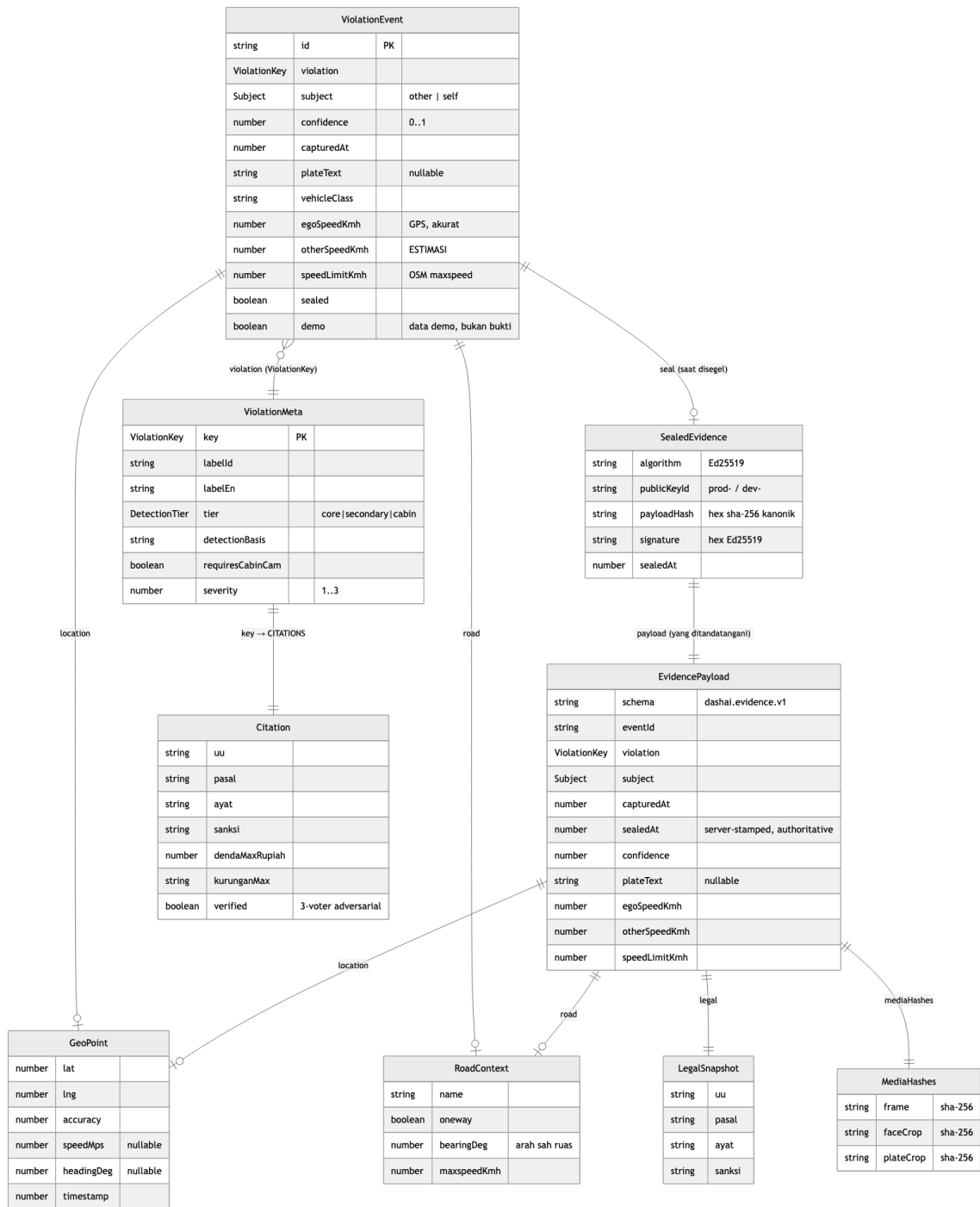
7.4 Pemicu otomatis kecelakaan

lib/sensors/motion.ts memantau DeviceMotion. Lonjakan akselerasi linear melampaui ambang (*HARD_BRAKE_G* = 0.75, *IMPACT_G* = 2.5, *debounce* 3 detik) memicu penangkapan bukti kecelakaan otomatis — sehingga footage sekitar tetap dapat disegel meski pengemudi tak dapat menyentuh ponsel.

8 Data view

8.1 Model domain & batas kriptografis

Tipe domain berpusat pada lib/evidence/types.ts. Batas kriptografis adalah EvidencePayload: *exact* JSON yang ditandatangani server. Segala sesuatu di luar payload bersifat presentasi dan **tidak** berbobot pembuktian.



Gambar 7: Entity-Relationship Diagram

8.1.1 EvidencePayload (yang ditandatangani)

Field	Tipe	Catatan
schema	"dashai.evidence.v1"	Versi skema payload
eventId	string	Identitas event
violation	ViolationKey	Salah satu dari 12 kunci pelanggaran
subject	"other" \ "self"	Subjek pelanggaran
capturedAt	number	Waktu tangkap klien
sealedAt	number	Distempel server (otoritatif)
confidence	number (0..1)	Di- <i>clamp</i> di server
vehicleClass, plateText	string / nullable	Atribut kendaraan
egoSpeedKmh	number / null	Akurat (GPS)
otherSpeedKmh	number / null	Estimasi (rentang galat)
speedLimitKmh	number / null	OSM maxspeed
location	GeoPoint / null	lat/lng, accuracy, speed, heading
road	RoadContext / null	oneway, bearingDeg, maxspeedKmh
mediaHashes	{ frame?, faceCrop?, plateCrop? }	SHA-256 byte media , bukan media-nya
legal	LegalSnapshot	uu, pasal, ayat, sanksi (snapshot KB)
device	{ userAgent?, platform? }	Opsional

Desain penting: payload **berkomitmen pada citra melalui hash** (mediaHashes), bukan menyertakan megabyte base64 — menjaga *signature* ringkas sekaligus mengikat bukti pada citranya.

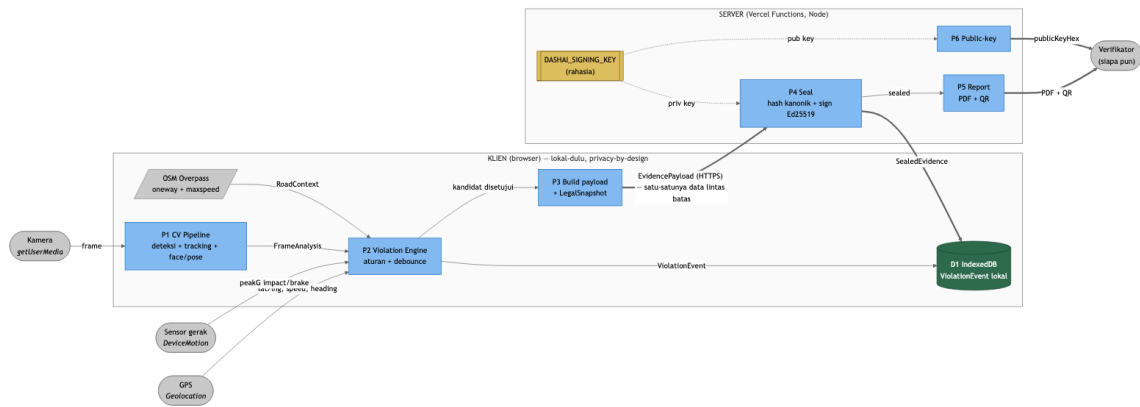
8.1.2 SealedEvidence (envelope tertandatangani)

{ payload, algorithm: "Ed25519", publicKeyId, payloadHash (hex SHA-256), signature (hex), sealedAt }. Inilah yang ditanam di laporan PDF, QR, dan URL /verify.

8.2 Penyimpanan & retensi

Tempat	Apa yang disimpan	Sifat
IndexedDB (dashai, v1)	ViolationEvent (termasuk seal bila tersegel) di store events, index by-time	<i>Local-first</i> , milik pemilik perangkat
Server Vercel	Tidak ada state event/bukti	<i>Stateless</i> ; verifikasi <i>self-contained</i>
Cache memori osm.ts	RoadContext per sel grid ~11m	<i>In-memory</i> , hilang saat reload
Env Vercel	DASHAI_SIGNING_KEY	Rahasia tunggal, server-only

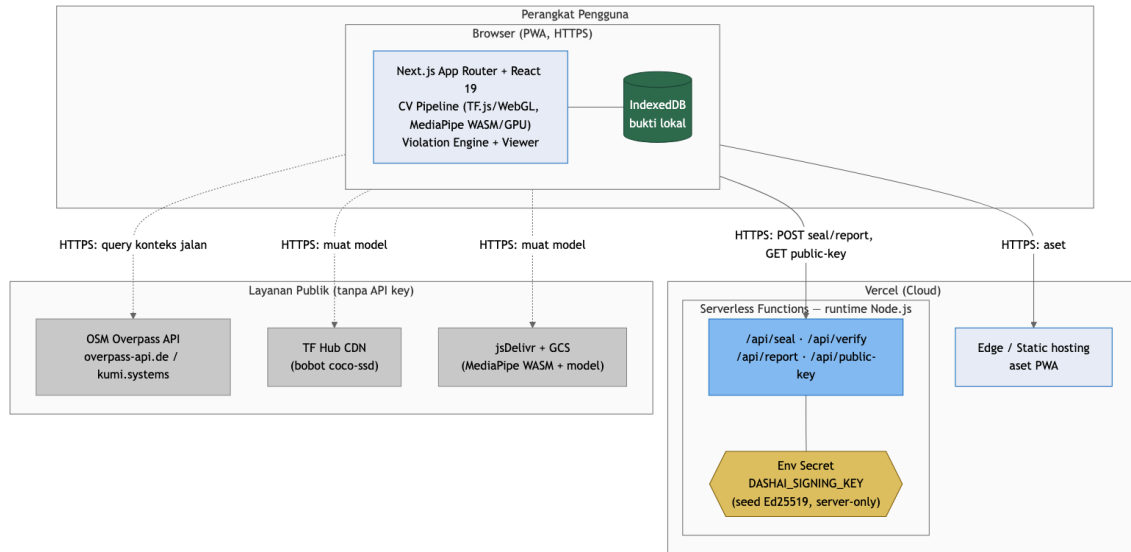
8.3 Aliran data & kanonikalisasi



Gambar 8: Data Flow Diagram

Kanonikalisasi (`lib/crypto/canonical.ts`) menjamin dua payload yang sama-secara-semantik menghasilkan *string* byte-identik: kunci objek diurutkan rekursif dan undefined dibuang. Inilah prasyarat agar hash + tanda tangan stabil lintas klien/server, sekaligus syarat agar `verifySealed` di peramban menghasilkan kesimpulan yang sama dengan server.

9 Deployment view (Vercel)



Gambar 9: Deployment

9.1 Topologi

Zona	Komponen	Keterangan
Perangkat Pengguna	Browser (PWA, HTTPS): Next.js App Router + React 19, CV Pipeline (TF.js/WebGL, MediaPipe WASM/GPU), Violation Engine + Viewer; IndexedDB	Seluruh inferensi & penyimpanan bukti di sini
Vercel (Cloud)	Edge / static hosting (aset PWA); Serverless Functions runtime Node.js (/api/seal, /api/verify, /api/report, /api/public-key); Env Secret DASHAI_SIGNING_KEY	Functions memerlukan Node (Ed25519, @react-pdf/renderer)
Layanan Publik (tanpa kunci)	OSM Overpass (overpass-api.de, overpass.kumi.systems); TF Hub CDN (coco-ssd); jsDelivr + GCS (MediaPipe WASM + model)	Dipanggil langsung dari peramban

9.2 Konfigurasi & rahasia

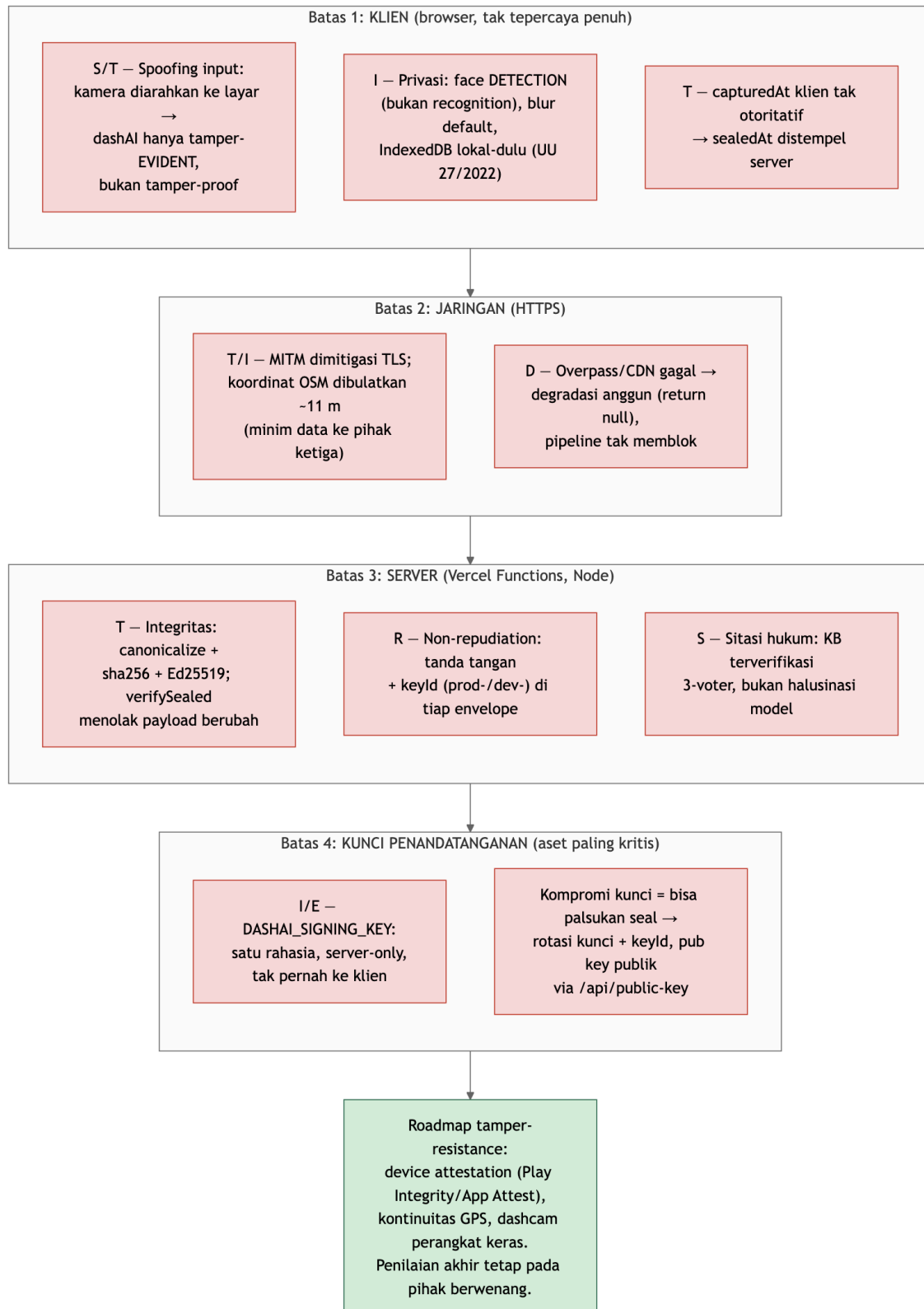
- **Runtime:** setiap *route* menyatakan `export const runtime = "nodejs"` — wajib untuk operasi kriptografis dan rendering PDF di *serverless*.
- **Kunci penandatanganan:** `DASHAI_SIGNING_KEY` (hex 64 karakter, seed `Ed25519` 32-byte). Tanpa env yang valid, `getServerKey` memakai **kunci DEV tetap** dan menandai `keyId` dengan prefiks `dev-`; *envelope* DEV tidak boleh dianggap otoritatif. Dengan env valid, prefiks `prod-`.
- **Bundling:** `next.config.ts` menjadikan `@react-pdf/renderer` *external* (`serverExternalPackages`) agar dependensi mirip-native (`fontkit`, `yoga`) tidak ikut di-*bundle* dan PDF dapat dirender di *serverless* Vercel.
- **Cache:** `/api/public-key` mengirim `Cache-Control: public, max-age=3600`.

9.3 Karakteristik *deployment*

- **Stateless:** tidak ada basis data; *scaling horizontal serverless* tanpa koordinasi *state*. Verifikasi tidak memerlukan *lookup* karena *envelope* lengkap menempel di laporan/QR/URL.
 - **Resiliensi eksternal:** Overpass memakai dua *endpoint* (gagal-alih); kegagalan jaringan menghasilkan `null` (tak memblok *pipeline*).
-

10 Cross-cutting concerns

10.1 Keamanan



Gambar 10: Threat model

Aspek	Mekanisme	Bukti kode
Integritas bukti	Ed25519 atas SHA-256 payload kanonik	lib/crypto/sign.ts, canonical.ts
Kerahasiaan kunci	<i>Private key</i> hanya di server (import "server-only"), tak pernah dikirim	sign.ts, keys.ts
Verifikasi tanpa kepercayaan UI	Kunci publik terbuka via /api/public-key; verifySealed jalan di klien	verify.ts, app/verify
Validasi input server	Tolak event tak lengkap / violation tak dikenal; <i>clamp</i> confidence	app/api/seal/route.ts
Anti-pemalsuan laporan	/api/report memverifikasi ulang, balas 422 bila tidak valid	app/api/report/route.ts
Batas jujur	<i>Tamper-evident</i> , bukan <i>tamper-proof</i> : tak ada <i>attestation</i> bahwa kamera melihat realitas	komentar sign.ts, README

Threat yang **belum** ditangani (jalan menuju produksi, dari README/komentar): *device attestation* (Play Integrity / App Attest), kontinuitas GPS, dan dashcam perangkat keras khusus untuk menaikkan biaya pemalsuan. Penilaian akhir tetap pada pihak berwenang.

10.2 Privasi

- **Deteksi, bukan pengenalan.** BlazeFace hanya melokalkan wajah untuk *crop* + blur; tidak ada pencocokan ke identitas (face.ts). Field faceCrop selalu di-blur di UI.
- **Local-first.** Bukti default tersimpan di IndexedDB; server hanya melihat payload yang **secara eksplisit** disegel (store.ts, komentar seal/route.ts).
- **Minimalisasi koordinat.** Lat/lng dibulatkan 4 desimal (~11 m) sebelum dikirim ke Overpass pihak ketiga (osm.ts).
- **Komitmen via hash.** Media direferensikan melalui SHA-256 di payload, bukan diunggah utuh.
- **Kepatuhan.** Selaras dengan semangat UU No. 27/2022 (PDP).

10.3 Performa

Teknik	Implementasi
Inferensi WebGL/GPU	TF.js <code>setBackend("webgl")</code> dengan fallback <code>cpu/wasm</code> (<code>detector.ts</code>); MediaPipe <code>delegate: "GPU"</code>
Model ringan	<code>coco-ssd</code> <code>lite_mobilenet_v2</code> ; Pose Landmarker <i>lite</i> ; BlazeFace <i>short-range</i>
Pembatasan inferensi mahal	OCR pelat hanya pada region terkonfirmasi, bukan tiap frame (<code>plate.ts</code>)
<i>Debounce</i>	Pelanggaran 6 detik (<code>engine.ts</code>); motion 3 detik (<code>motion.ts</code>)
Gerbang <i>noise</i>	<code>MIN_TRACK_AGE = 4</code> , ambang flow, minimal 3 kendaraan bergerak
Caching jaringan	RoadContext per grid ~11 m; kunci publik <code>max-age=3600</code>
Degradasi anggun	Model sekunder <i>try/catch</i> ; live view tetap jalan
<i>Lazy import</i>	Pustaka berat (<code>@tensorflow/tfjs</code> , <code>tasks-vision</code> , <code>tesseract.js</code>) di- <i>import</i> dinamis

FPS, jumlah deteksi, dan status pipeline dilaporkan ke `useSession` untuk dipantau di HUD.

10.4 Observability

- **Status pipeline:** `PipelineStatus { object, face, pose }` dengan callback `onProgress` saat *load* model.
- **Metrik sesi:** `fps`, `detectionCount`, `geo/road/egoSpeedKmh` di `lib/state/session.ts`.
- **Sinyal verifikasi:** `VerificationResult { valid, reason, hashMatches, signatureValid, publicKeyId, checkedAt }` memberi alasan terbaca-manusia (Bahasa Indonesia) saat VALID/DIUBAH.
- **Indikator lingkungan:** `isDev` pada kunci publik & laporan menandai mode DEV agar tak disalahartikan otoritatif.

Catatan kejujuran: `dashAI` sengaja **tidak** mengirim telemetri/penyimpanan terpusat (konsekuensi *local-first*); observabilitas bersifat sisi-klien.

11 Technology decisions & rationale

Keputusan	Alternatif	Rasional
Next.js 16 (App Router) + React 19	SPA murni, framework lain	Satu <i>codebase</i> untuk UI + <i>serverless API</i> ; deploy mulus di Vercel; <i>server-only</i> untuk batas kripto
TypeScript strict	JS	Tipe domain (EvidencePayload) sebagai kontrak batas kripto; mencegah kebocoran field tak tertandatangani
Inferensi di peramban (TF.js + MediaPipe + Tesseract.js)	Backend GPU	<i>Privacy-by-design</i> (tanpa upload video), tanpa biaya GPU, <i>edge-native</i>
coco-ssd lite_mobilenet_v2 di balik ObjectDetector	YOLOv8/ONNX sekarang	Cepat di ponsel; antarmuka memungkinkan <i>swap</i> ke YOLOv8/ONNX tanpa ubah pipeline
IoU tracker buatan	DeepSORT/ByteTrack	Cukup untuk velocity + id stabil; ringan, tanpa model tambahan
MediaPipe BlazeFace (deteksi)	Pengenalan wajah	Hanya untuk blur; menegaskan deteksi bukan pengenalan
Ed25519 via @noble/ed25519 + @noble/hashes	RSA, library besar	Tanda tangan ringkas/cepat; <i>verifyAsync</i> jalan identik di klien & server (Web Crypto SHA-512)
JSON kanonik kunci-tersortir	JCS eksternal	Determinisme byte minimal-dependensi untuk hash/signature stabil
Satu env secret (DASHAI_SIGNING_KEY)	KMS/HSM	Permukaan rahasia minimal; cukup untuk demo; <i>roadmap</i> ke <i>attestation</i>
OpenStreetMap Overpass (tanpa kunci)	Google Roads (berbayar)	<i>oneway/maxspeed</i> gratis; dua endpoint gagal-alih; privasi via pembulatan
IndexedDB via idb	localStorage, server DB	<i>Local-first</i> , kapasitas besar, milik pengguna
@react-pdf/renderer, runtimanpdf.js + qr code	HTML → PDF eksternal Redragon	Render PDF di <i>serverless</i> ; <i>signature</i> minimal; OR membawa <i>envelope</i> untuk verifikasi <i>self-contained</i> Mono) cepat dibangun

12 Quality scenarios

Skenario kualitas berikut mengikuti gaya *quality attribute scenario* (sumber, stimulus, lingkungan, artefak, respons, ukuran respons), dipetakan ke mekanisme arsitektur nyata.

12.1 QS-1 Integritas (Security)

Stimulus: Penyerang mengubah satu *byte* (mis. mengganti pelat/kecepatan/pasal) pada laporan tersegel. **Respons:** Saat diverifikasi, `recomputedHash !== payloadHash` → `hashMatches = false` → hasil **DIUBAH** dengan alasan “Hash payload tidak cocok — isi laporan telah diubah setelah disegel.” **Ukuran:** 100% perubahan isi terdeteksi; tak ada *false negative* selama kanonikalisasi konsisten.

12.2 QS-2 Kemandirian verifikasi (Security/Trust)

Stimulus: Verifikator tidak memercayai UI dashAI. **Respons:** Mengambil kunci publik via `/api/public-key` dan menjalankan `verifySealed` sepenuhnya di peramban (`/verify`). **Ukuran:** Verifikasi berhasil tanpa memanggil `/api/verify`; kepercayaan hanya pada kunci publik + algoritme.

12.3 QS-3 Privasi koordinat (Privacy)

Stimulus: Pengguna khawatir lokasi presisi bocor ke pihak ketiga. **Respons:** `osm.ts` membulatkan lat/lng ke 4 desimal sebelum kueri Overpass. **Ukuran:** Presisi yang dikirim $\leq \sim 11$ m; koordinat penuh tak pernah meninggalkan perangkat untuk konteks jalan.

12.4 QS-4 Ketahanan jaringan (Availability)

Stimulus: Endpoint Overpass utama tak responsif saat berkendara. **Respons:** Gagal-alih ke endpoint kedua; bila keduanya gagal, kembalikan `null` dan *cache* hasil; *pipeline* tetap berjalan. **Ukuran:** Live view tidak terblokir; nol *exception* tak tertangani dari jaringan.

12.5 QS-5 Robustness model (Reliability)

Stimulus: Model wajah/pose gagal *load* di peramban tertentu. **Respons:** `CVPipeline` membungkus dengan `try/catch`; `status.face/pose` tetap `false`; deteksi objek + aturan tetap jalan. **Ukuran:** Fungsi inti (deteksi pelanggaran *live*) tetap tersedia.

12.6 QS-6 Akurasi yang jujur (Correctness)

Stimulus: Pengguna ingin tahu seberapa dipercaya angka kecepatan. **Respons:** `egoSpeedKmh` (GPS) ditandai akurat; `otherSpeedKmh` ditandai estimasi; aturan *self* hanya memicu di atas `limit + 5 km/jam`. **Ukuran:** Pemisahan akurat/estimasi terkode di tipe & katalog; *speeding-self* memakai sumber akurat.

12.7 QS-7 Provenans hukum (Maintainability/Trust)

Stimulus: Reviewer hukum mempertanyakan pasal yang dikutip. **Respons:** Sitasi berasal dari KB terverifikasi 3-voter; di-*snapshot* ke payload tertandatangani; artefak riset tersedia. **Ukuran:** 12/12 sitasi `verified: true`, `confidence: high`; pasal pada laporan = pasal pada CITATIONS.

12.8 QS-8 Anti-pemalsuan laporan (Security)

Stimulus: Klien mengirim *envelope* tak valid ke `/api/report`. **Respons:** Server memverifikasi ulang; bila tidak valid, balas 422 tanpa membuat PDF. **Ukuran:** Tak ada laporan “resmi” dihasilkan untuk *envelope* yang gagal verifikasi.

12.9 QS-9 Minimnya jejak waktu palsu (Integrity)

Stimulus: Klien mencoba memalsukan waktu penyegelan. **Respons:** `/api/seal` mengabaikan waktu klien dan menstempel `sealedAt = Date.now()` di server, lalu menandatangani. **Ukuran:** Waktu tertandatangani selalu otoritatif-server.

12.10 QS-10 Pluggability (Modifiability)

Stimulus: Tim menambah pelanggaran baru / menukar detektor. **Respons:** Tambah entri `VIOLATION_CATALOG + CITATIONS` + aturan `engine.ts`; atau implementasikan ulang `ObjectDetector`. **Ukuran:** Perubahan terlokalisasi; *pipeline* dan batas krypto tak tersentuh.

13 Lampiran — Ringkasan keterbatasan demo

- dashAI adalah **demonstrasi teknologi**; laporan **bukan** dokumen resmi kepolisian dan sitasi bersifat indikatif.
- **Tamper-evident**, bukan *tamper-proof* — tak ada *attestation* bahwa kamera menyaksikan realitas.
- Deteksi *live*: lawan arah (other/self), ngebut diri sendiri (GPS vs OSM), boncengan lebih, terobos lampu merah (self). Pelanggaran lain dikatalogkan + ditampilkan via demo.
- Estimasi kecepatan kendaraan lain bersifat perkiraan (rentang galat).
- Jalan menuju produksi: *device attestation*, *backend* model khusus, kamera kabin, kalibrasi estimasi, integrasi ETLE resmi.